《机器学习基础》

前馈神经网络

https://github.com/styluck/mech_learn

内容

- 神经网络
  - 神经元
  - 网络结构
- 前馈神经网络
  - 参数学习
  - 计算图与自动微分
  - 优化问题

神经网络

神经网络

- 神经网络最早是作为一种主要的连接主义模型。
  - 20世纪80年代后期，最流行的一种连接主义模型是分布式并行处理（Parallel Distributed Processing，PDP）网络，其有3个主要特性：
    - 1）信息表示是分布式的（非局部的）；
    - 2）记忆和知识是存储在单元之间的连接上；
    - 3）通过逐渐改变单元之间的连接强度来学习新的知识。

  - 引入**误差反向传播算法**来改进其学习能力之后，神经网络也越来越多地应用在各种机器学习任务上。

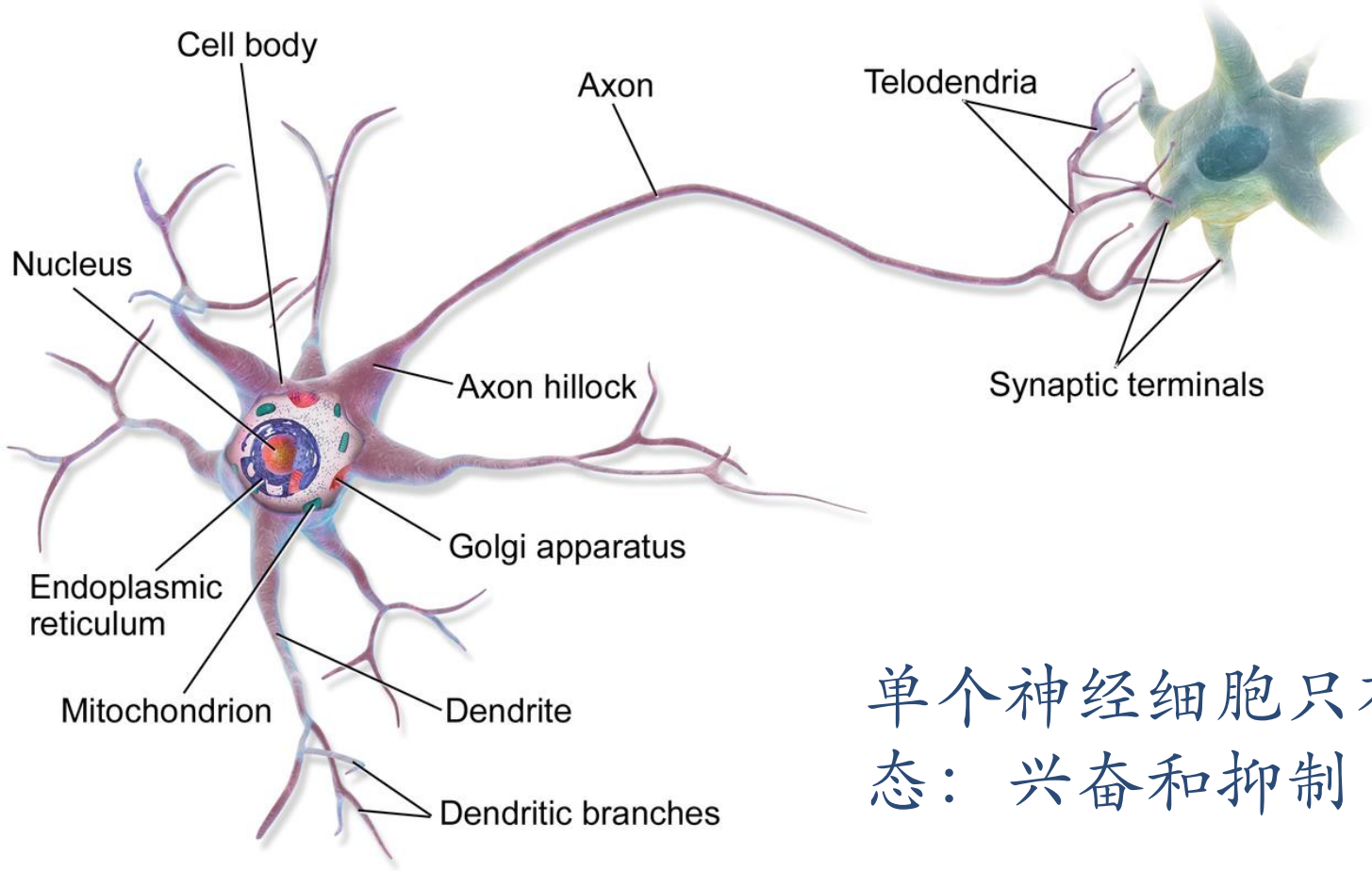
神经网络

- 分布式设计的好处
  - 编码冗余
    - 容错性
  - 组合能力
    - 计算效率
  - 抽象和泛化
    - 灵活性和扩展性

神经元

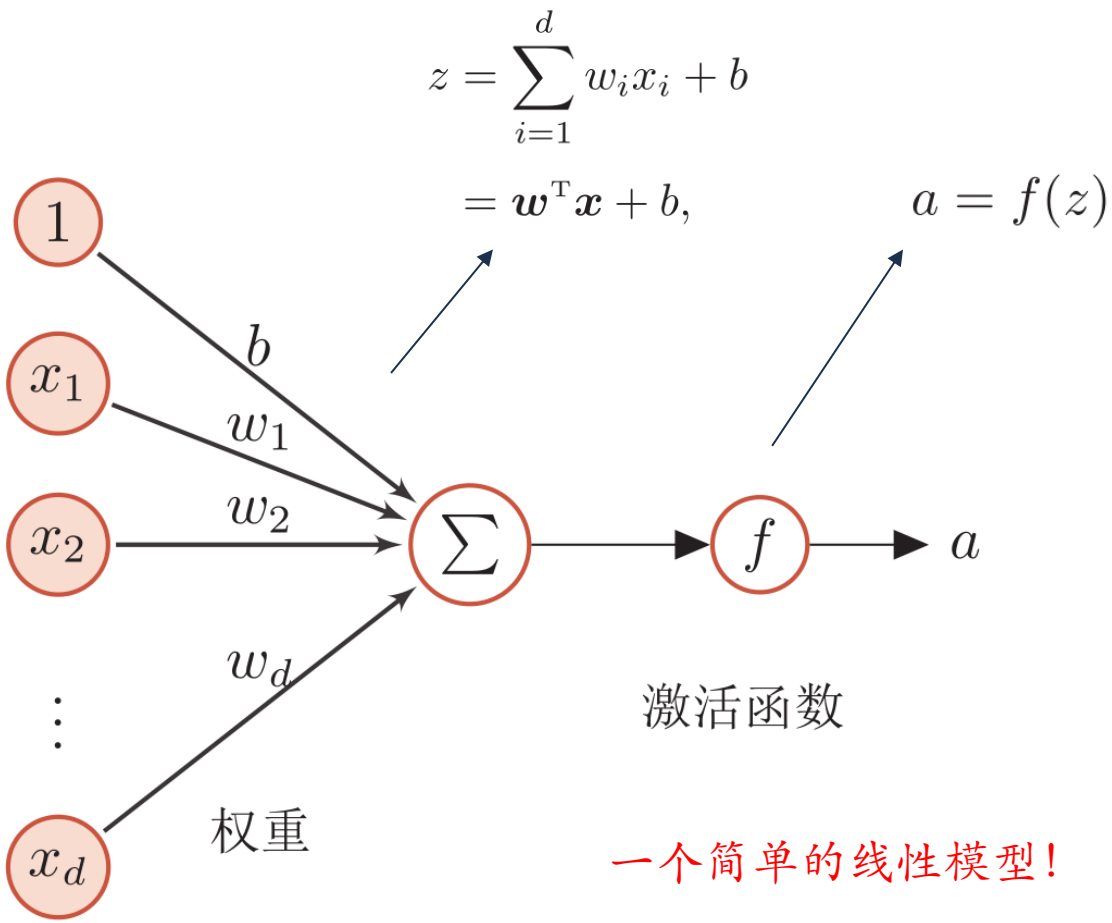
生物神经元

video: structure of brain



单个神经细胞只有两种状态：兴奋和抑制

人工神经元



一个简单的线性模型！

激活函数的性质

- 连续并可导（允许少数点上不可导）的非线性函数。
  - 可导的激活函数可以直接利用数值优化的方法来学习网络参数。
- 激活函数及其导函数要尽可能的简单
  - 有利于提高网络计算效率。
- 激活函数的导函数的值域要在一个合适的区间内
  - 不能太大也不能太小，否则会影响训练的效率和稳定性。
- 单调性
  - 具有单调性的激活函数（如ReLU）可以确保网络学习到的权重在输入空间中保持一致的正向或负向影响，这有助于网络捕捉到更复杂的非线性关系。
  - 并不是所有的神经网络函数都需要单调性。

常见激活函数

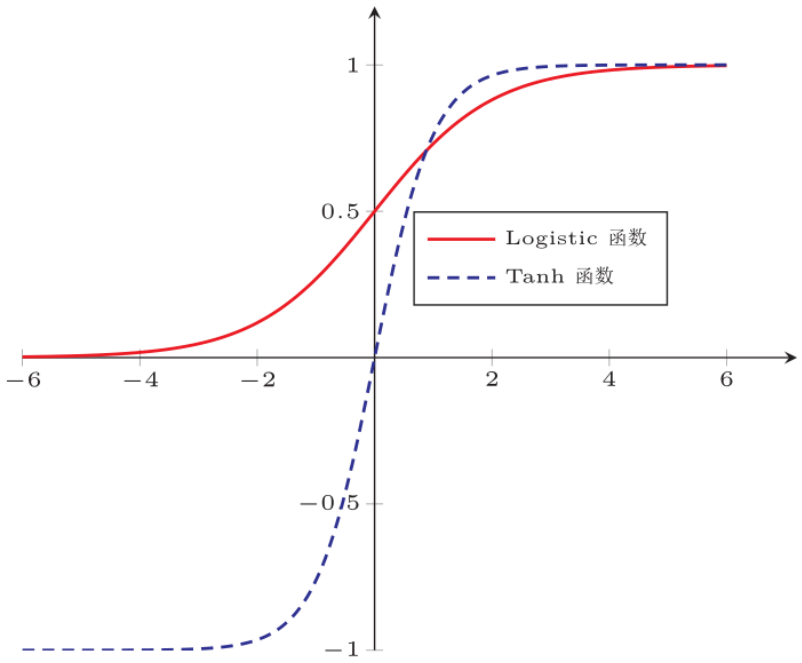
- logistic函数

$$\sigma(x) = \frac{1}{1+\exp(-x)}$$

  - 输出恒大于0
  - 非零中心化
- Tanh函数

$$\tanh(x) = \frac{\exp(x)-\exp(-x)}{\exp(x)+\exp(-x)}$$

  - 饱和函数
  - 零中心化



非零中心化的输出会使得其后一层的神经元的输入发生偏置偏移（bias shift），并进一步使得梯度下降的收敛速度变慢。

常见激活函数

- ReLU函数

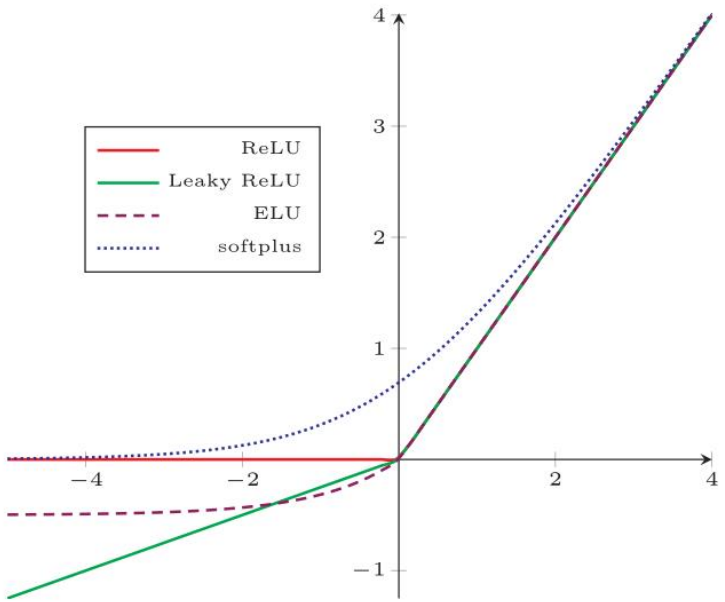
$$\mathrm{ReLU}(x) = \begin{cases} x & x \geq 0 \\ 0 & x < 0 \end{cases}$$
$$= \max(0, x).$$

- 类ReLU函数

$$\mathrm{ELU}(x) = \begin{cases} x & \text{if } x > 0 \\ \gamma(\exp(x) - 1) & \text{if } x \leq 0 \end{cases}$$
$$= \max(0, x) + \min(0, \gamma(\exp(x) - 1))$$

$$\mathrm{LeakyReLU}(x) = \begin{cases} x & \text{if } x > 0 \\ \gamma x & \text{if } x \leq 0 \end{cases}$$
$$= \max(0, x) + \gamma \min(0, x)$$

$$\mathrm{softplus}(x) = \log(1 + \exp(x))$$



- 计算上更加高效
- 生物学合理性
  - 单侧抑制、宽兴奋边界
- 在一定程度上缓解梯度消失问题
- 死亡ReLU问题（Dying ReLU Problem）

常见激活函数及其导数

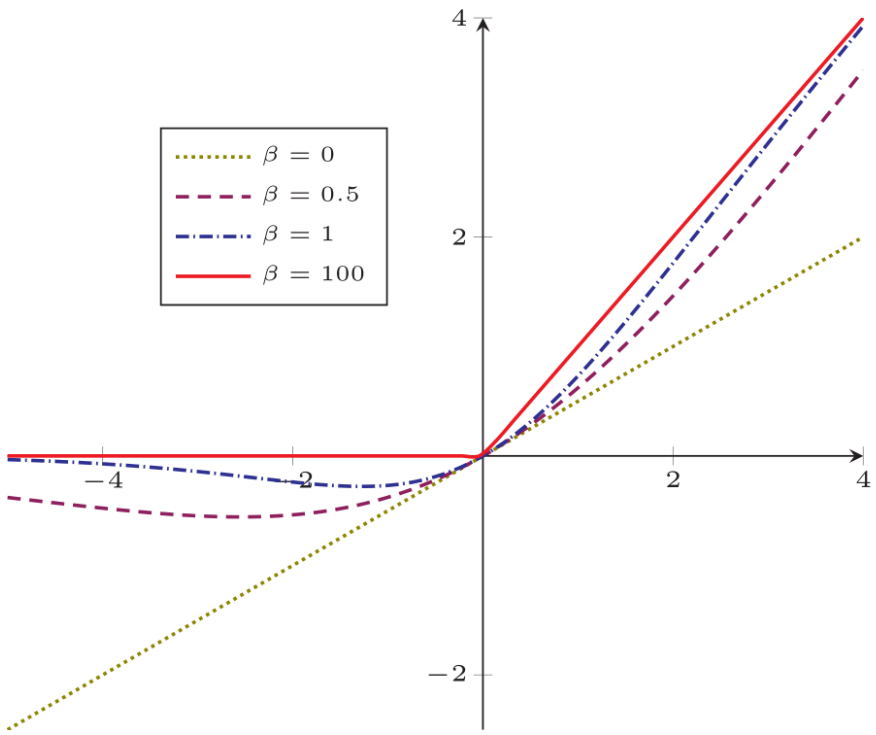
激活函数	函数	导数
Logistic 函数	$f(x) = \frac{1}{1+\exp(-x)}$	$f'(x) = f(x)\big(1 - f(x)\big)$
Tanh 函数	$f(x) = \frac{\exp(x)-\exp(-x)}{\exp(x)+\exp(-x)}$	$f'(x) = 1 - f(x)^2$
ReLU 函数	$f(x) = \max(0, x)$	$f'(x) = I(x > 0)$
ELU 函数	$f(x) = \max(0, x) + \min\Big(0, \gamma\big(\exp(x) - 1\big)\Big)$	$f'(x) = I(x > 0) + I(x \leq 0) \cdot \gamma \exp(x)$
SoftPlus 函数	$f(x) = \log\big(1 + \exp(x)\big)$	$f'(x) = \frac{1}{1+\exp(-x)}$

常见激活函数

Swish函数

$$\mathrm{swish}(x) = x\sigma(\beta x)$$

$$\sigma(x) = \frac{1}{1+\exp(-x)}$$



常见激活函数

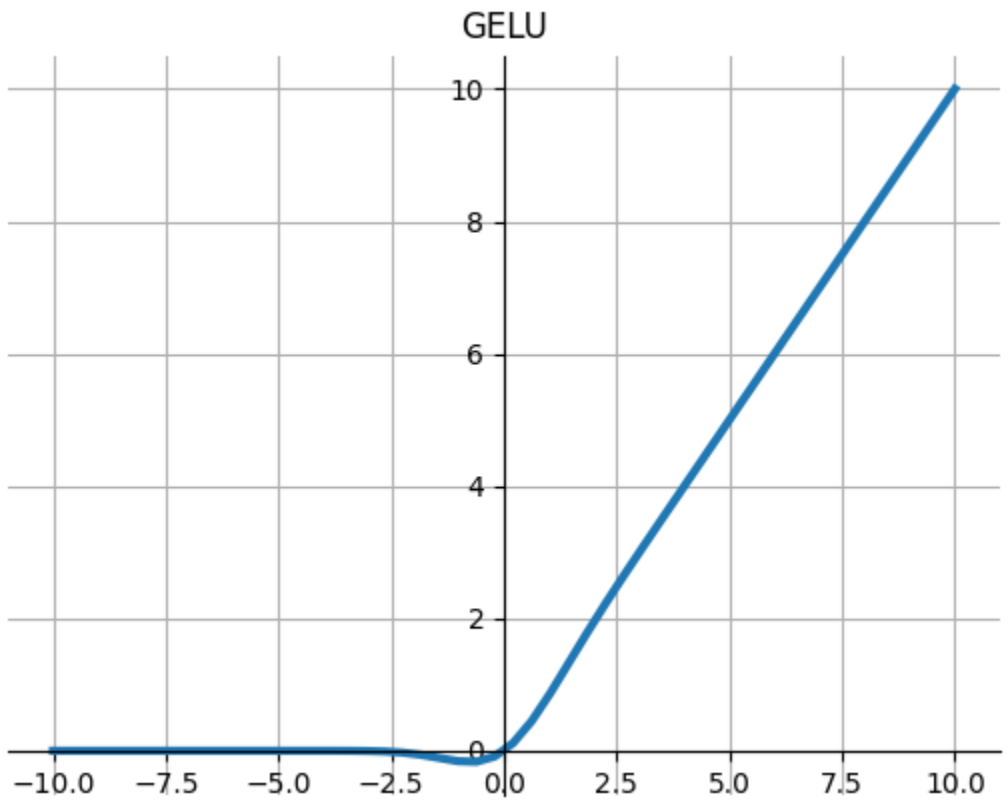
- 高斯误差线性单元（Gaussian Error Linear Unit，GELU）

$$\mathrm{GELU}(x) = xP(X \le x)$$

  - 其中P(X ≤ x)是高斯分布N(μ, σ^2)的累积分布函数，其中μ,σ为超参数，一般设μ = 0,σ = 1即可
  - 由于高斯分布的累积分布函数为S型函数，因此GELU可以用Tanh函数或Logistic函数来近似

$$\mathrm{GELU}(x) \approx 0.5x\left(1 + \tanh\left(\sqrt{\frac{2}{\pi}}(x + 0.044715x^3)\right)\right)$$

或 $$\mathrm{GELU}(x) \approx x\sigma(1.702x).$$



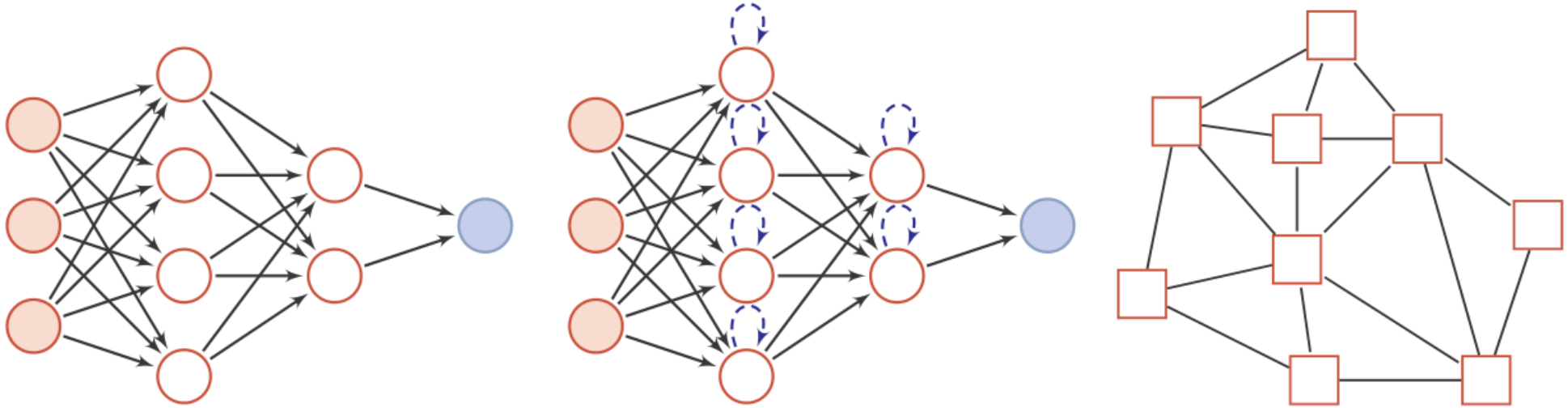
如何构建人工神经网络

- 人工神经网络主要由大量的神经元以及它们之间的有向连接构成。因此考虑三方面：

- 神经元的激活规则
  - 主要是指神经元输入到输出之间的映射关系，一般为非线性函数。
- 网络的拓扑结构
  - 不同神经元之间的连接关系。
- 学习算法
  - 通过训练数据来学习神经网络的参数。

网络结构

- 人工神经网络由神经元模型构成，这种由许多神经元组成的信息处理网络具有并行分布结构。



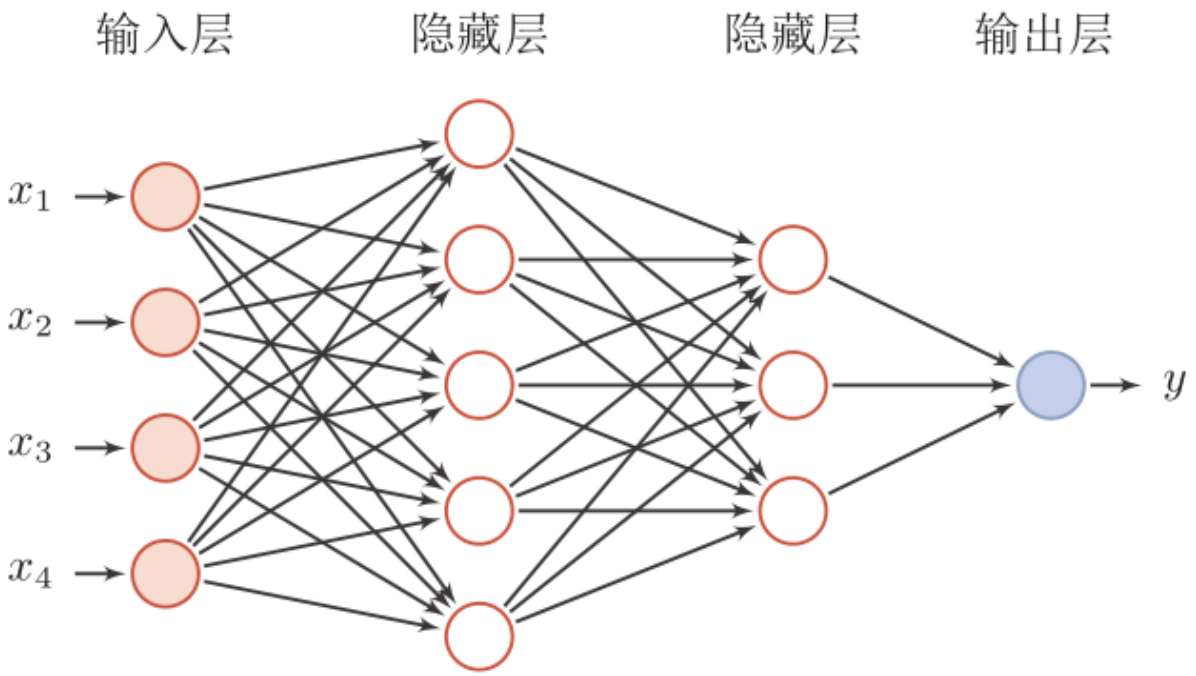
(a) 前馈网络　(b) 记忆网络　(c) 图网络

圆形节点表示一个神经元，方形节点表示一组神经元。

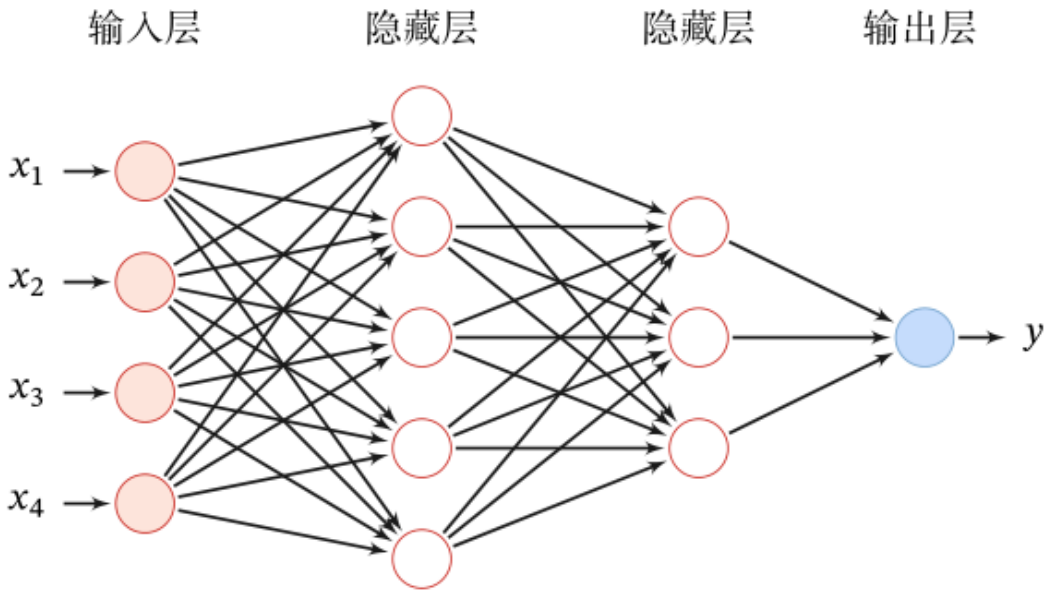
前馈神经网络

网络结构

- 前馈神经网络（**全连接**神经网络、多层感知器）
  - 各神经元分别属于不同的层，层内无连接。
  - 相邻两层之间的神经元全部两两连接。
  - 整个网络中无反馈，信号从输入层向输出层单向传播，可用一个有向无环图表示。



前馈网络



给定一个前馈神经网络，用下面的记号来描述这样网络：

记号	含义
L	神经网络的层数
M_l	第 l 层神经元的个数
$f_l(\cdot)$	第 l 层神经元的激活函数
$\boldsymbol{W}^{(l)} \in \mathbb{R}^{M_l \times M_{l-1}}$	第 $l-1$ 层到第 l 层的权重矩阵
$\boldsymbol{b}^{(l)} \in \mathbb{R}^{M_l}$	第 $l-1$ 层到第 l 层的偏置
$\boldsymbol{z}^{(l)} \in \mathbb{R}^{M_l}$	第 l 层神经元的净输入（净活性值）
$\boldsymbol{a}^{(l)} \in \mathbb{R}^{M_l}$	第 l 层神经元的输出（活性值）

应用到机器学习

- 神经网络可以作为一个“万能”函数来使用，可以用来进行复杂的特征转换，或逼近一个复杂的条件分布。

$$\hat{y} = g(\underline{\varphi(\mathbf{x})}, \theta)$$

分类器　　神经网络

- 如果$g(\cdot)$为Logistic回归，那么Logistic回归分类器可以看成神经网络的最后一层。

通用近似定理

定理 4.1 – 通用近似定理（Universal Approximation Theorem）[Cybenko, 1989, Hornik et al., 1989]： 令 $\varphi(\cdot)$ 是一个非常数、有界、单调递增的连续函数，$\mathcal{I}_d$ 是一个 d 维的单位超立方体 $[0,1]^d$，$C(\mathcal{I}_d)$ 是定义在 $\mathcal{I}_d$ 上的连续函数集合。对于任何一个函数 $f \in C(\mathcal{I}_d)$，存在一个整数 m，和一组实数 $v_i, b_i \in \mathbb{R}$ 以及实数向量 $\mathbf{w}_i \in \mathbb{R}^d$，$i = 1, \cdots, m$，以至于我们可以定义函数

$$F(\mathbf{x}) = \sum_{i=1}^{m} v_i \varphi(\mathbf{w}_i^{\mathrm{T}} \mathbf{x} + b_i), \tag{4.33}$$

作为函数 f 的近似实现，即

$$|F(\mathbf{x}) - f(\mathbf{x})| < \epsilon, \forall \mathbf{x} \in \mathcal{I}_d. \tag{4.34}$$

其中 $\epsilon > 0$ 是一个很小的正数。

- 根据通用近似定理
  - 对于具有线性输出层和至少一个使用“挤压”性质的激活函数的隐藏层组成的前馈神经网络。
  - 只要其隐藏层神经元的数量足够，它可以以任意的精度来近似任何一个定义在实数空间中的有界闭集函数。

参数学习

基于梯度下降法的参数学习

- 给定训练集为$D = \{(x_i, y_i)\}_{i=1}^{N}$，将每个样本$x_i$输入给神经网络，得到网络输出为$\hat{y}_i$，其在数据集D上的风险函数为：

$$R(\omega, b) = \frac{1}{N}\sum_{i=1}^{N} L(\hat{y}_i, y_i)$$

- 我们需要进行学习的参数有ω和b，梯度下降算法的迭代格式：

$$w \leftarrow w - \alpha \frac{dR(w, b)}{dw}$$

$$b \leftarrow b - \alpha \frac{dR(w, b)}{db}$$

优化算法：随机梯度下降

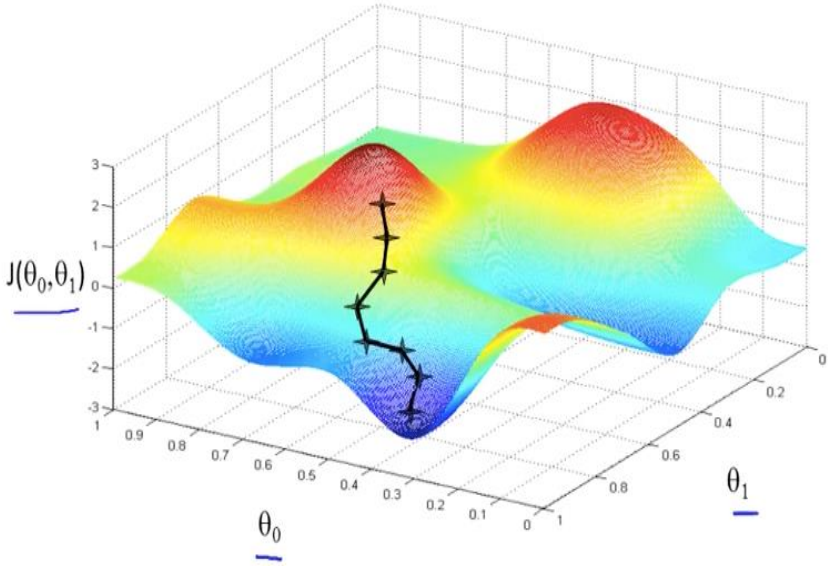
算法 2.1　随机梯度下降法

输入: 训练集 $\mathcal{D} = \{(\boldsymbol{x}^{(n)}, y^{(n)})\}_{n=1}^{N}$，验证集 $\mathcal{V}$，学习率 α

```
随机初始化 θ;
repeat
    对训练集 D 中的样本随机排序;
    for n = 1 ··· N do
        从训练集 D 中选取样本 (x^(n), y^(n));
        θ ← θ − α ∂L(θ; x^(n), y^(n)) / ∂θ;
    end
until 模型 f(x; θ) 在验证集 V 上的错误率不再下降;
```

$$\theta \leftarrow \theta - \alpha \frac{\partial \mathcal{L}(\theta; x^{(n)}, y^{(n)})}{\partial \theta};$$

输出: θ



反向传播算法

前向传播：信息的传递过程

- 前馈神经网络通过下面公式进行信息传播。

$$\boldsymbol{z}^{(l)} = \boldsymbol{W}^{(l)}\boldsymbol{a}^{(l-1)} + \boldsymbol{b}^{(l)},$$
$$\boldsymbol{a}^{(l)} = f_l(\boldsymbol{z}^{(l)}).$$

- 前向计算：

$$\boldsymbol{x} = \boldsymbol{a}^{(0)} \to \boldsymbol{z}^{(1)} \to \boldsymbol{a}^{(1)} \to \boldsymbol{z}^{(2)} \to \cdots \to \boldsymbol{a}^{(L-1)} \to \boldsymbol{z}^{(L)} \to \boldsymbol{a}^{(L)} = \phi(\boldsymbol{x}; \boldsymbol{W}, \boldsymbol{b}))$$

如何计算梯度?

- 神经网络为一个复杂的复合函数
  - 链式法则

$$y = f^5(f^4(f^3(f^2(f^1(x))))) \rightarrow \frac{\partial y}{\partial x} = \frac{\partial f^1}{\partial \mathrm{x}}\frac{\partial f^2}{\partial f^1}\frac{\partial f^3}{\partial f^2}\frac{\partial f^4}{\partial f^3}\frac{\partial f^5}{\partial f^4}$$

- 反向传播算法
  - 根据前馈网络的特点而设计的高效方法
  - 反向传播算法(Back-Propagation, BP)，深度学习核心之一
  - 加速计算参数梯度值的方法

矩阵微积分

- 矩阵微积分（Matrix Calculus）是多元微积分的一种表达方式，即使用矩阵和向量来表示因变量每个成分关于自变量每个成分的偏导数。
  - 若$y = f(x) \in R, x \in R^p$标量关于向量的偏导数：

$$\frac{\partial y}{\partial x} = \frac{\partial f(x)}{\partial x} = \left[\frac{\partial f}{\partial x_1}, \frac{\partial f}{\partial x_2}, \dots, \frac{\partial f}{\partial x_p}\right]^T \in R^p,$$

  - 若$y = f(x) \in R^q, x \in R^p$向量关于向量的偏导数（雅可比矩阵）：

$$\frac{\partial y}{\partial x} = \frac{\partial f(x)}{\partial x} = \begin{bmatrix} \left(\frac{\partial f_1(x)}{\partial x}\right)^T \\ \vdots \\ \left(\frac{\partial f_q(x)}{\partial x}\right)^T \end{bmatrix} = \left[\frac{\partial f_i(x)}{\partial x_j}\right] \in R^{q \times p},$$

链式法则

- 链式法则（Chain Rule）是在微积分中求复合函数导数的一种常用方法。
  - 若 $x \in R$，$\boldsymbol{u} = \boldsymbol{u}(x) \in R^s$，$\boldsymbol{g} = \boldsymbol{g}(u) \in R^t$，则
$$\frac{\partial \boldsymbol{g}}{\partial x} = \frac{\partial \boldsymbol{g}}{\partial \boldsymbol{u}} \frac{\partial \boldsymbol{u}}{\partial x} \in R^t.$$
  - (2) 若 $\boldsymbol{x} \in R^p$，$\boldsymbol{g} = \boldsymbol{g}(\boldsymbol{x}) \in R^s$，$\boldsymbol{f} = \boldsymbol{f}(\boldsymbol{g}) \in R^t$，则
$$\frac{\partial \boldsymbol{f}}{\partial \boldsymbol{x}} = \frac{\partial \boldsymbol{f}}{\partial \boldsymbol{g}} \frac{\partial \boldsymbol{g}}{\partial \boldsymbol{x}} \in R^{t \times p}.$$
  - (3) 若 $X \in R^{p \times q}$ 为矩阵，$\boldsymbol{g} = \boldsymbol{g}(X) \in R^s$，$f = f(\boldsymbol{g}) \in R$，则
$$\frac{\partial f}{\partial X} = \left(\frac{\partial f}{\partial \boldsymbol{g}}\right)\left(\frac{\partial \boldsymbol{g}}{\partial X}\right) = \sum_{k=1}^{s} \frac{\partial f}{\partial g_k} \frac{\partial g_k}{\partial X} \in R^{p \times q}.$$

计算图和自动微分

- 自动微分是利用链式法则来自动计算一个复合函数的梯度。

$$f(x; w, b) = \frac{1}{\exp(-(wx + b)) + 1}.$$

- 计算图

计算图和自动微分

- 当$x = 1, w = 0, b = 0$时，可得

计算图和自动微分

```
class Matmul:
    def __init__(self):
        self.mem = {}

    def forward(self, x, W):
        h = np.matmul(x, W)
        self.mem={'x': x, 'W':W}
        return h

    def backward(self, grad_y):
        x = self.mem['x']
        W = self.mem['W']

        grad_W = np.matmul(x.T,grad_y)
        grad_x = np.matmul(grad_y,W.T)

        return grad_x, grad_W
```

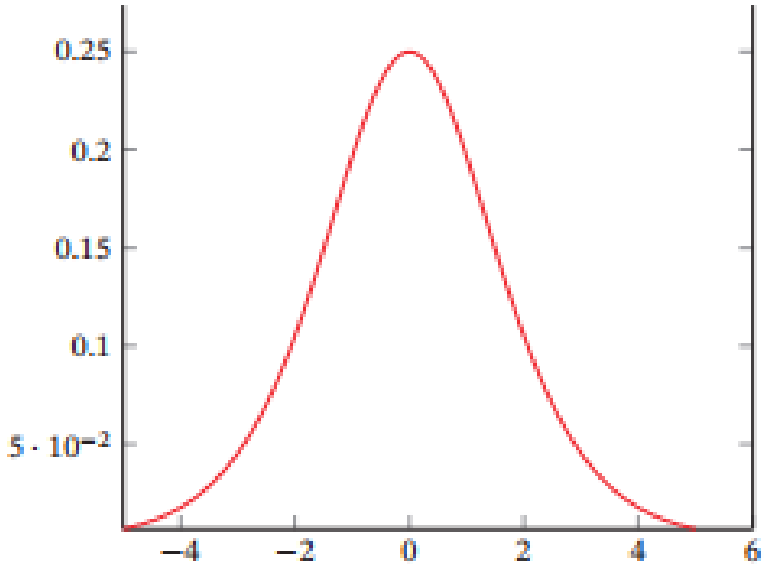
梯度消失和梯度爆炸

- 梯度消失:
  - （1）隐藏层的层数过多；
  - （2）采用了不合适的激活函数(更容易产生梯度消失，但是也有可能产生梯度爆炸)
  - （3）在深度神经网络中， 减轻梯度消失问题的方法有很多种． 一种简单有效的方式是使用导数比较大的激活函数， 比如ReLU等．

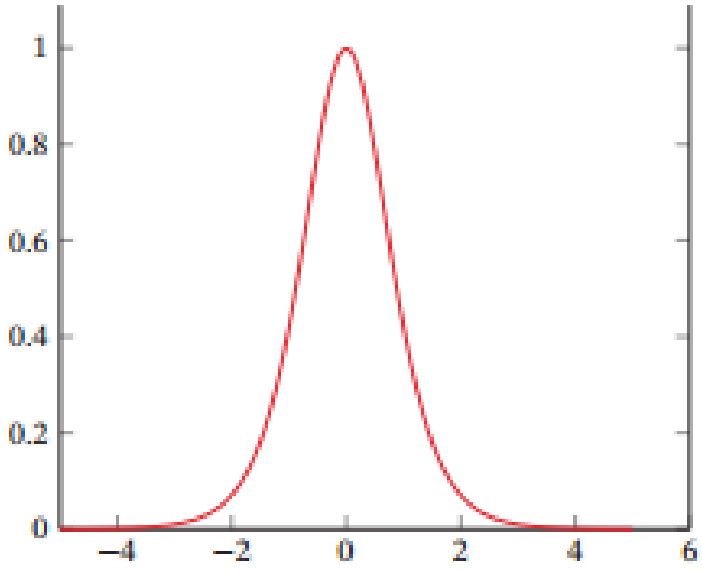
- 梯度爆炸:
  - （1）隐藏层的层数过多；
  - （2）权重的初始化值过大

梯度消失

- 由于 Sigmoid 型函数的饱和性，饱和区的导数更是接近于 0． 这样，误差经过每一层传递都会不断衰减．当网络层数很深时，梯度就会不停衰减，甚至消失，使得整个网络很难训练．



(a) Logistic 函数的导数

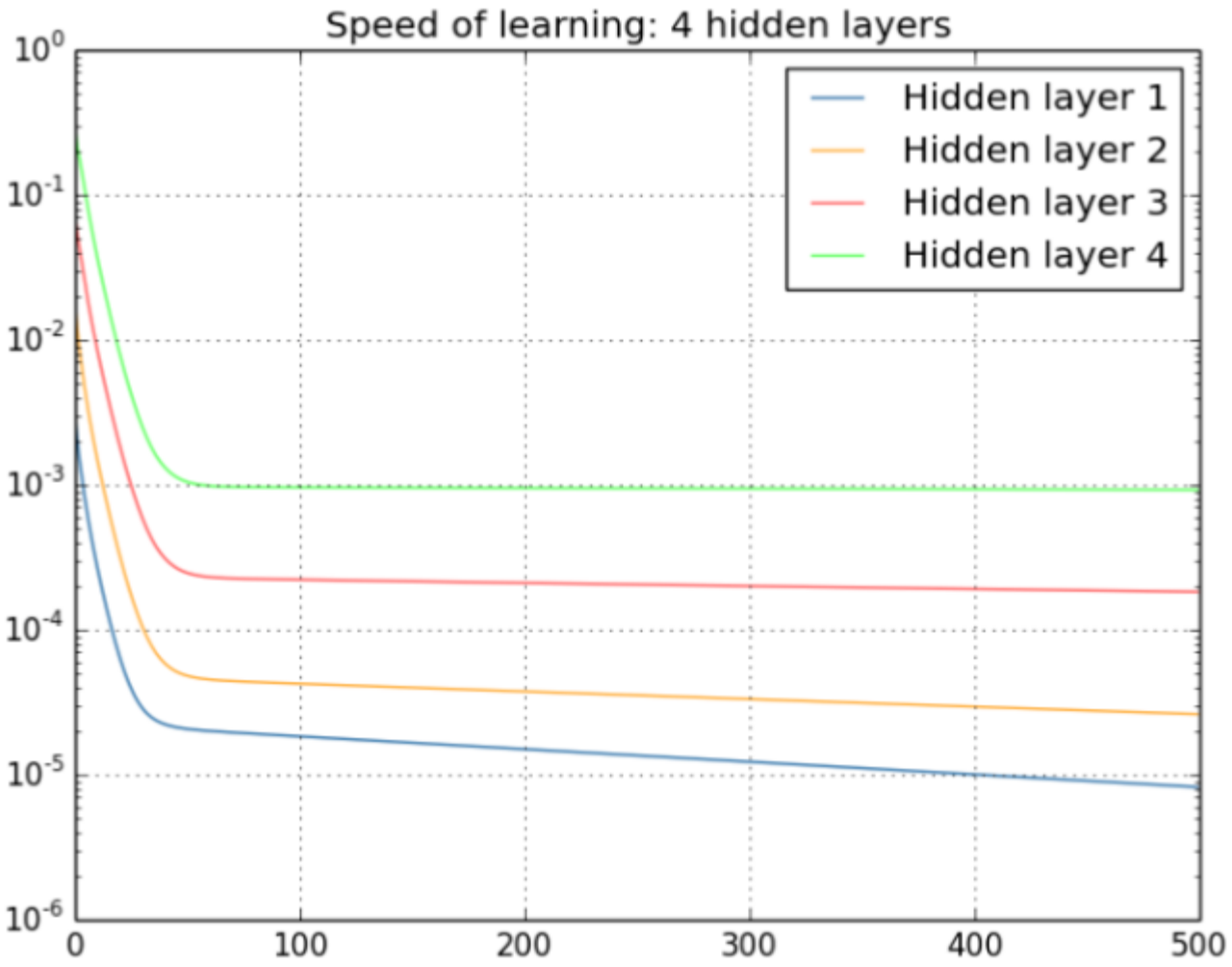


(b) Tanh 函数的导数

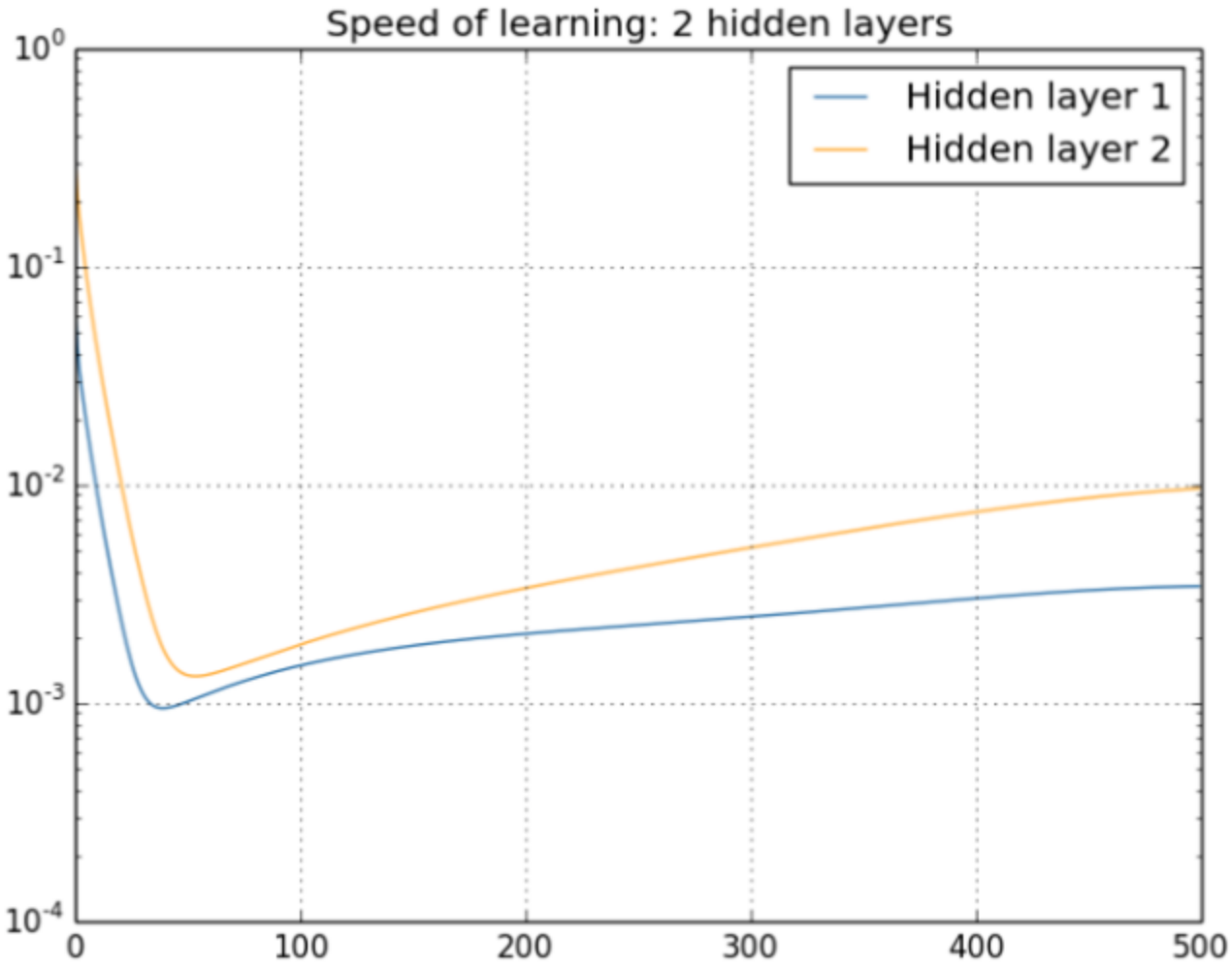
$$y = f^5(f^4(f^3(f^2(f^1(x)))))$$

$$\frac{\partial y}{\partial x} = \frac{\partial f^1}{\partial \mathrm{x}}\frac{\partial f^2}{\partial f^1}\frac{\partial f^3}{\partial f^2}\frac{\partial f^4}{\partial f^3}\frac{\partial f^5}{\partial f^4}$$

梯度消失


Speed of learning: 4 hidden layers
Hidden layer 1
Hidden layer 2
Hidden layer 3
Hidden layer 4
10^{0}
10^{-1}
10^{-2}
10^{-3}
10^{-4}
10^{-5}
10^{-6}
0
100
200
300
400
500

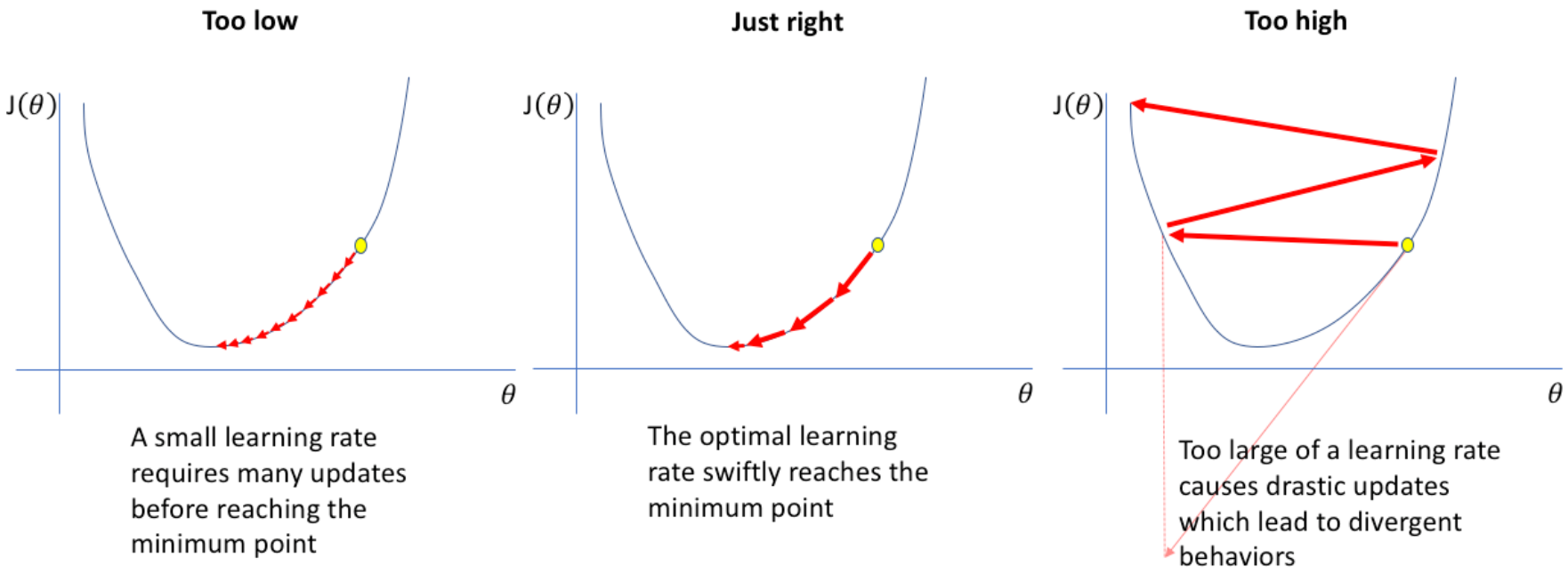
梯度爆炸


Speed of learning: 2 hidden layers
Hidden layer 1
Hidden layer 2
10^{0}
10^{-1}
10^{-2}
10^{-3}
10^{-4}
0
100
200
300
400
500

学习率调整

学习率的影响

$$\theta \leftarrow \theta - \alpha g$$

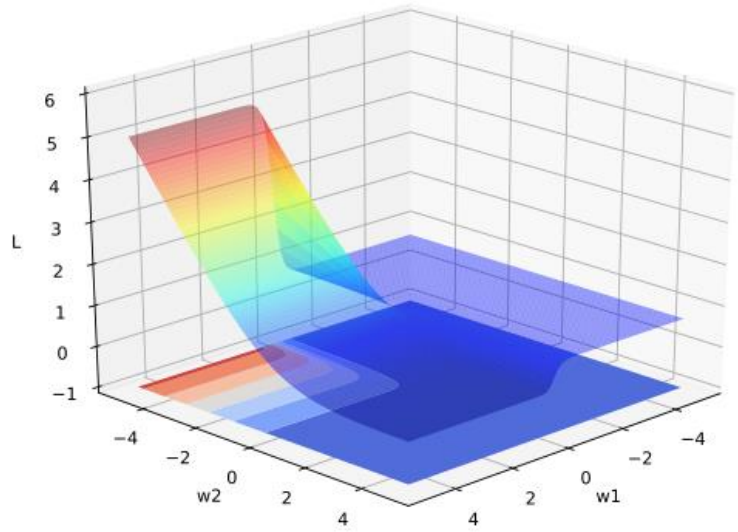


https://www.jeremyjordan.me/nn-learning-rate/

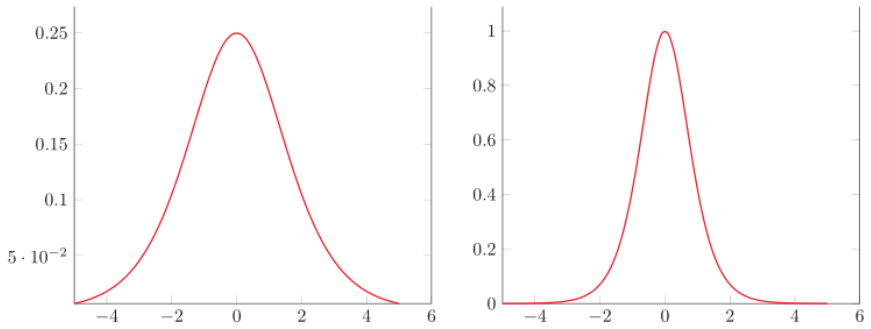
演示

优化问题

- 非凸优化问题
  - $y = \sigma(w_2\sigma(w_1x))$的损失函数

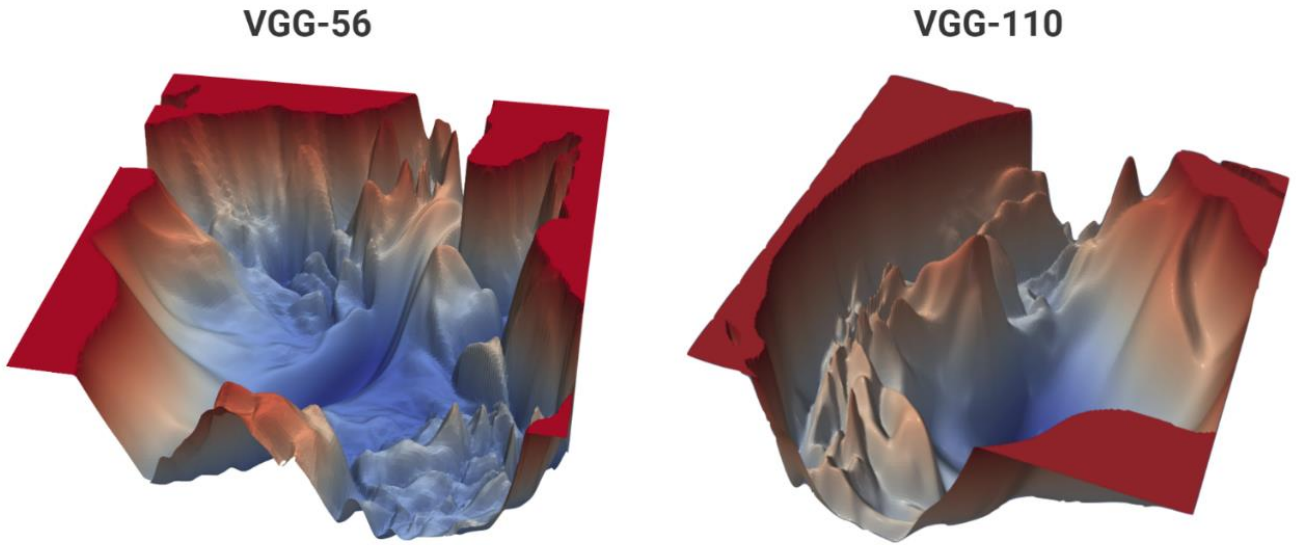


- 梯度消失问题
  - 在每一层都要乘以该层的激活函数的导数

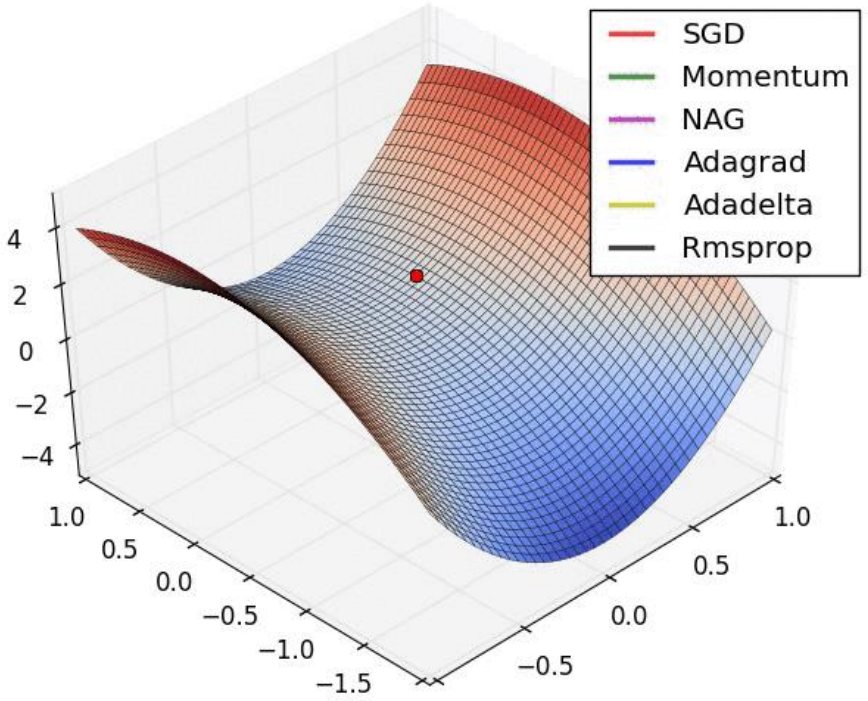


(a) logistic 函数的导数　(b) tanh 函数的导数

高维空间的非凸优化问题



问题2：陷入鞍点（saddle point）



问题1：陷入局部最优解

学习率衰减

假设初始化学习率为α_0，在第t次迭代时的学习率α_t。常用的衰减方式为可以设置为按迭代次数进行衰减。比如逆时衰减（inverse time decay）

$$\alpha_t = \alpha_0 \frac{1}{1 + \beta \times t},$$

或指数衰减（exponential decay）

$$\alpha_t = \alpha_0 \beta^t,$$

或自然指数衰减（natural exponential decay）

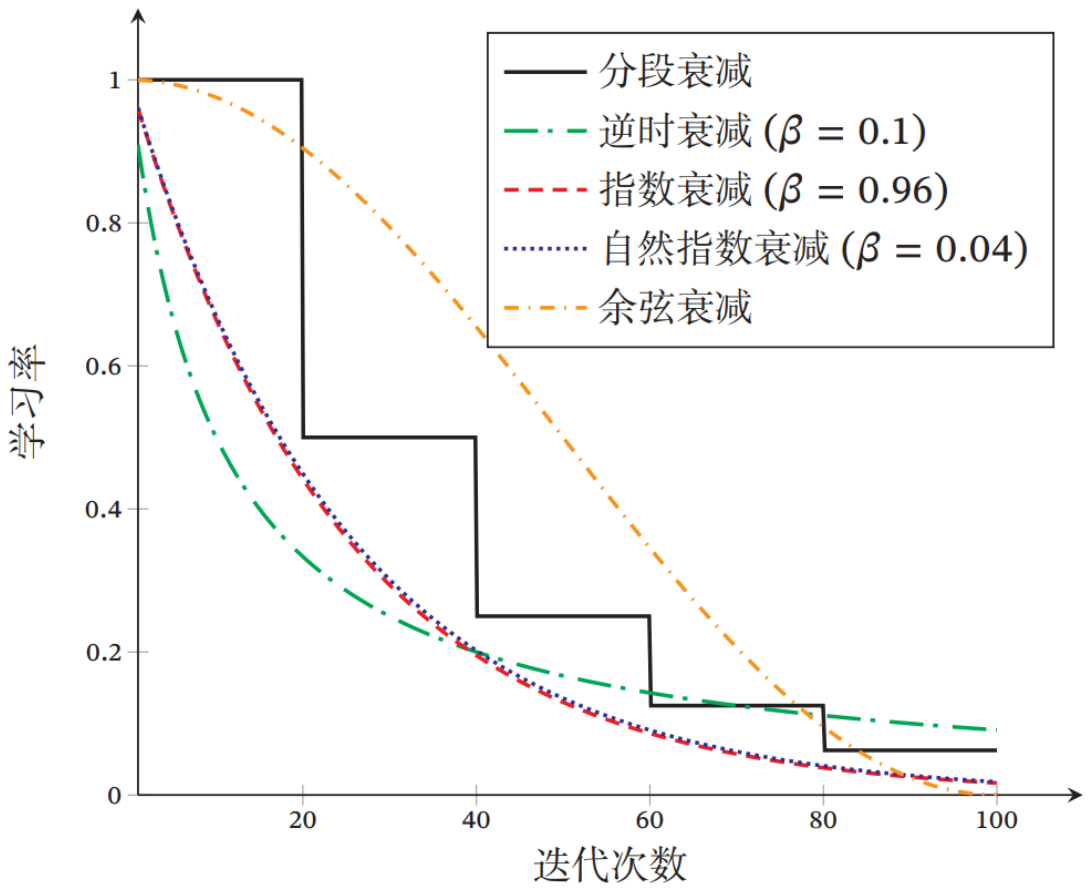
$$\alpha_t = \alpha_0 \exp(-\beta \times t),$$

其中β为衰减率，一般取值为0.96。

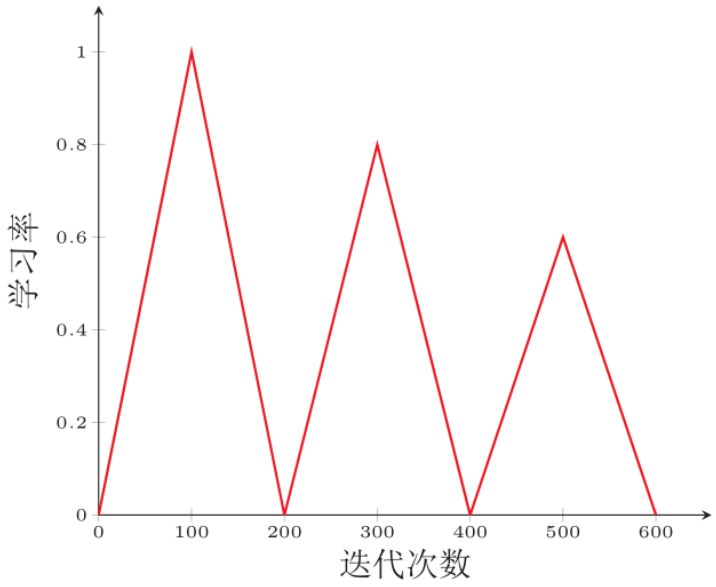
学习率衰减

$$g(\theta) = \frac{1}{K}\nabla_\theta \sum_{i=1}^{K} L\left(f\left(x_i, \theta\right), y_i\right)$$

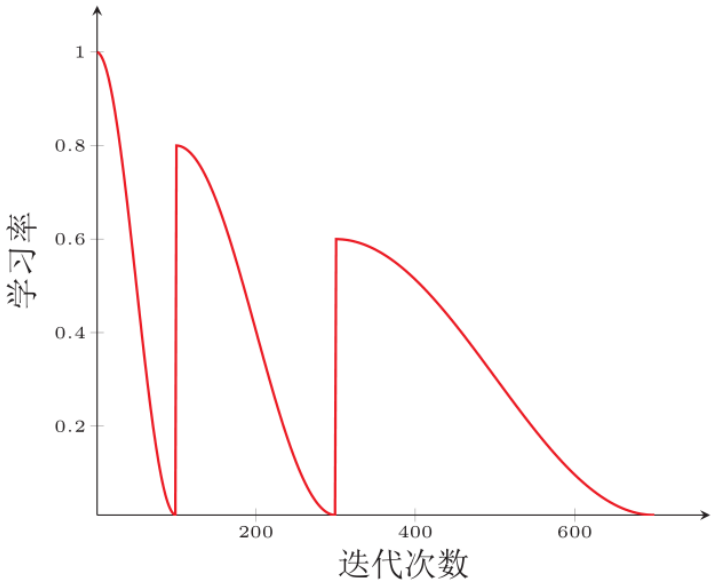
$$\theta \leftarrow \theta - \alpha g$$



周期性学习率调整 Cyclical Learning Rates

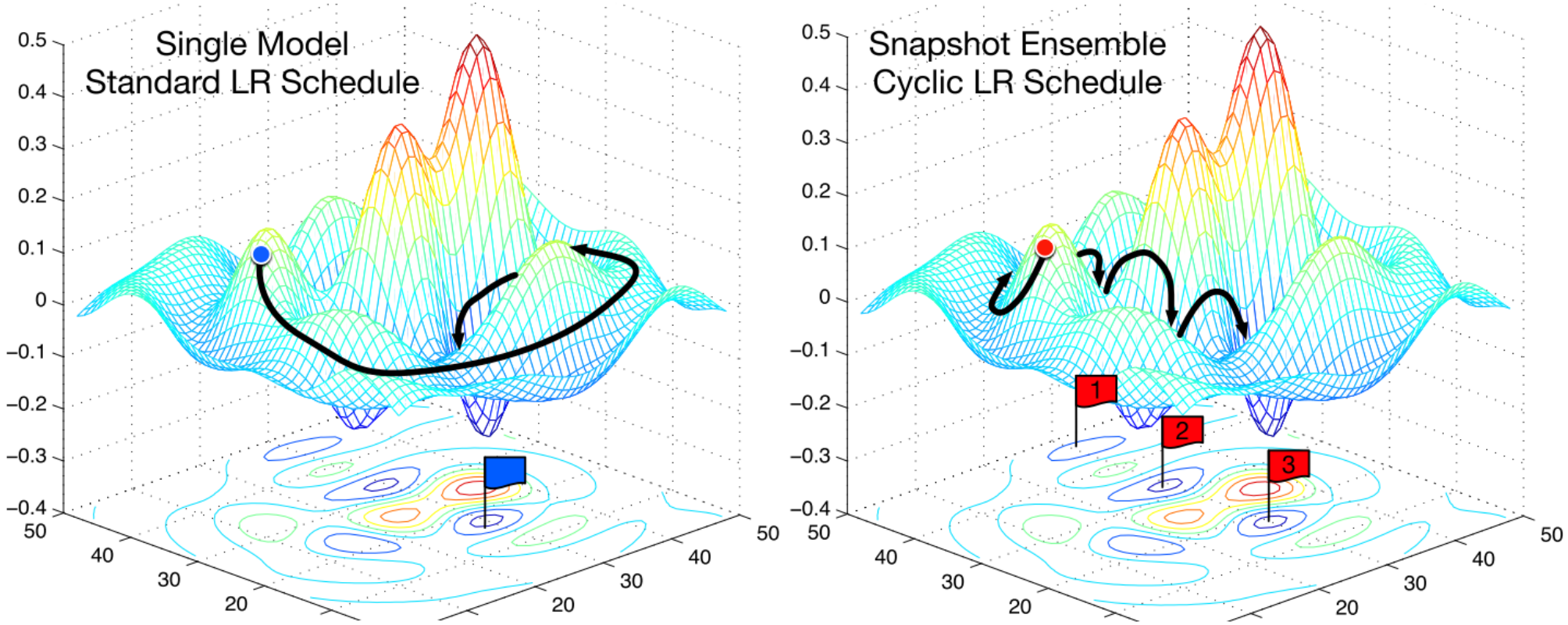


(a) 三角循环学习率



(b) 带热重启的余弦衰减

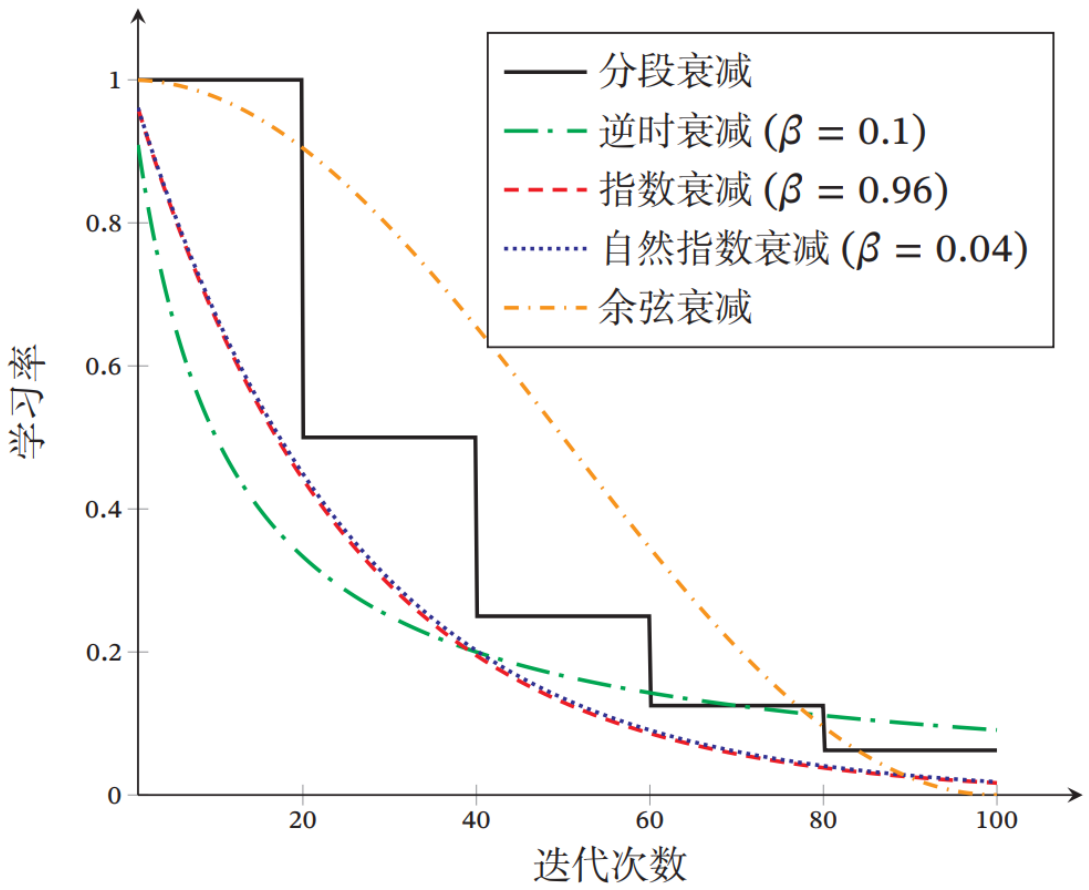
Cyclical Learning Rates



学习率衰减

$$g(\theta) = \frac{1}{K}\nabla_\theta \sum_{i=1}^{K} L\left(f\left(x_i, \theta\right), y_i\right)$$

$$\theta \leftarrow \theta - \alpha g$$



AdaGrad和RMSProp

- AdaGrad(2011)：每次迭代时自适应地调整每个参数的学习率。

- 在第t次迭代时，先计算每个参数分量梯度平方的累计值:

- 参数更新差值为:

- 从而得到新的参数:

ε 是为了保持数值稳定性而设置的非常小的常数。

$$g(\theta) = \frac{1}{K}\nabla_\theta \sum_{i=1}^{K} L\left(f\left(x_i, \theta\right), y_i\right)$$

$$G_t = \sum_{\tau=1}^{t} g_\tau \odot g_\tau$$

$$\Delta\theta_t = -\frac{\alpha}{\sqrt{G_t + \varepsilon}} \odot g_t$$

$$\theta \leftarrow \theta - \frac{\alpha}{\sqrt{G_t + \varepsilon}} \odot g_t$$

AdaGrad和RMSProp

- RMSProp(2012)：避免 AdaGrad 算法中学习率不断单调下降以至于过早衰减的缺点。
- 在第t次迭代时，先计算每个参数分量梯度平方的累计值:

$$g(\theta) = \frac{1}{K}\nabla_\theta \sum_{i=1}^{K} L\left(f\left(x_i, \theta\right), y_i\right)$$

$$G_t = \beta G_{t-1} + (1-\beta)\, g_t \odot g_t$$

- 参数更新差值为:

$$\Delta\theta_t = -\frac{\alpha}{\sqrt{G_t + \varepsilon}} \odot g_t$$

- 从而得到新的参数:

$$\theta \leftarrow \theta - \frac{\alpha}{\sqrt{G_t + \varepsilon}} \odot g_t$$

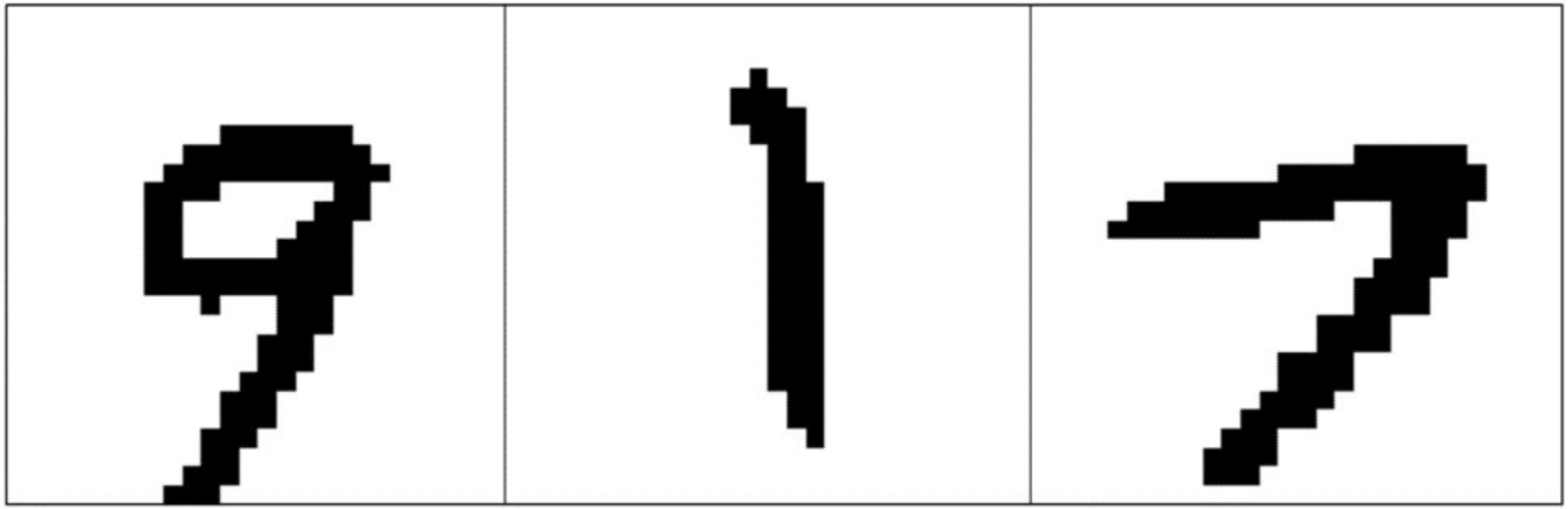
ε 是为了保持数值稳定性而设置的非常小的常数。

MNIST数据库训练

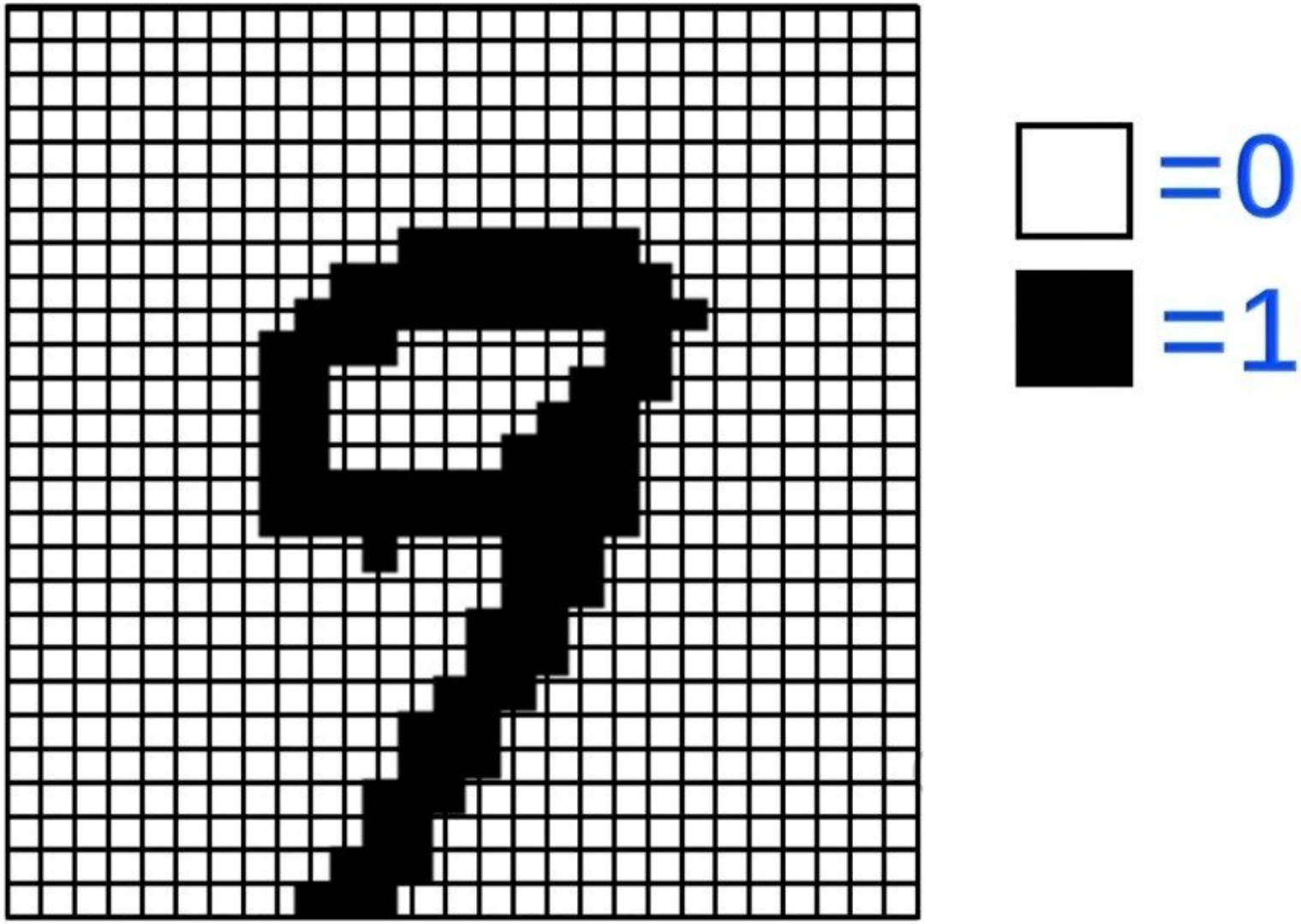
MNIST数据库

- MNIST数据库，全称是Modified National Institute of Standards and Technology database，即修改后的国家标准与技术研究所数据库。
- 包含70,000个手写数字的灰度图像，训练集 60000 张图像用于训练，测试集10000 张。
- 这些图像大小统一为28x28像素。
- 每个图像都是通过对原始图像进行中心化、缩放和去噪等预处理操作得到的。

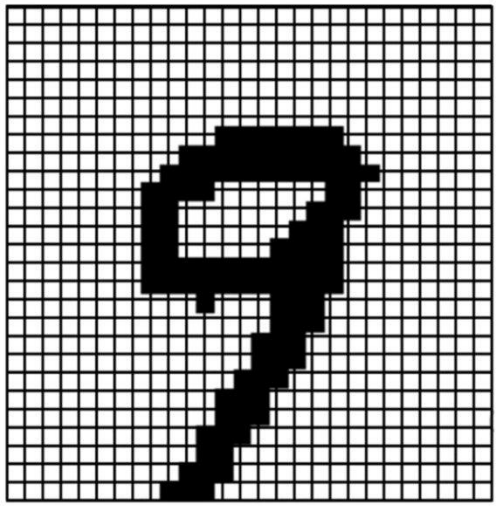
MNIST数据库



MNIST数据库



手写数字训练

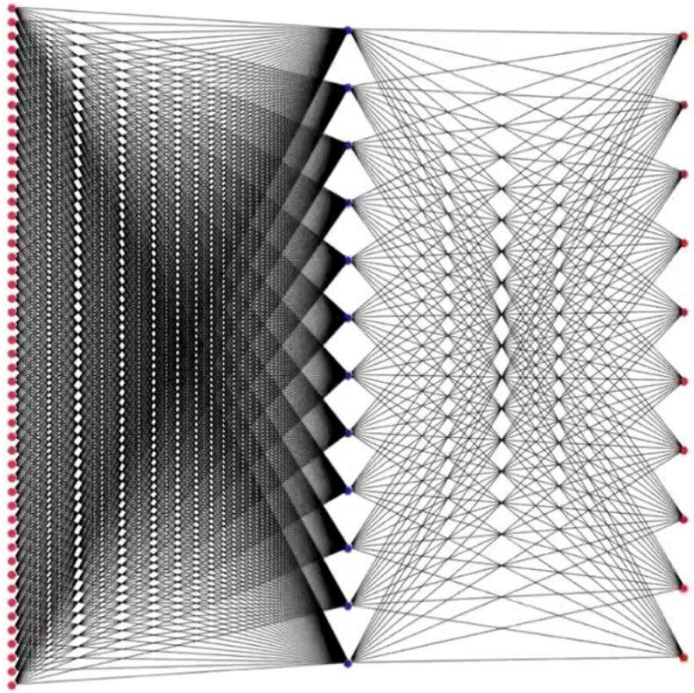


$n \times n$

=0

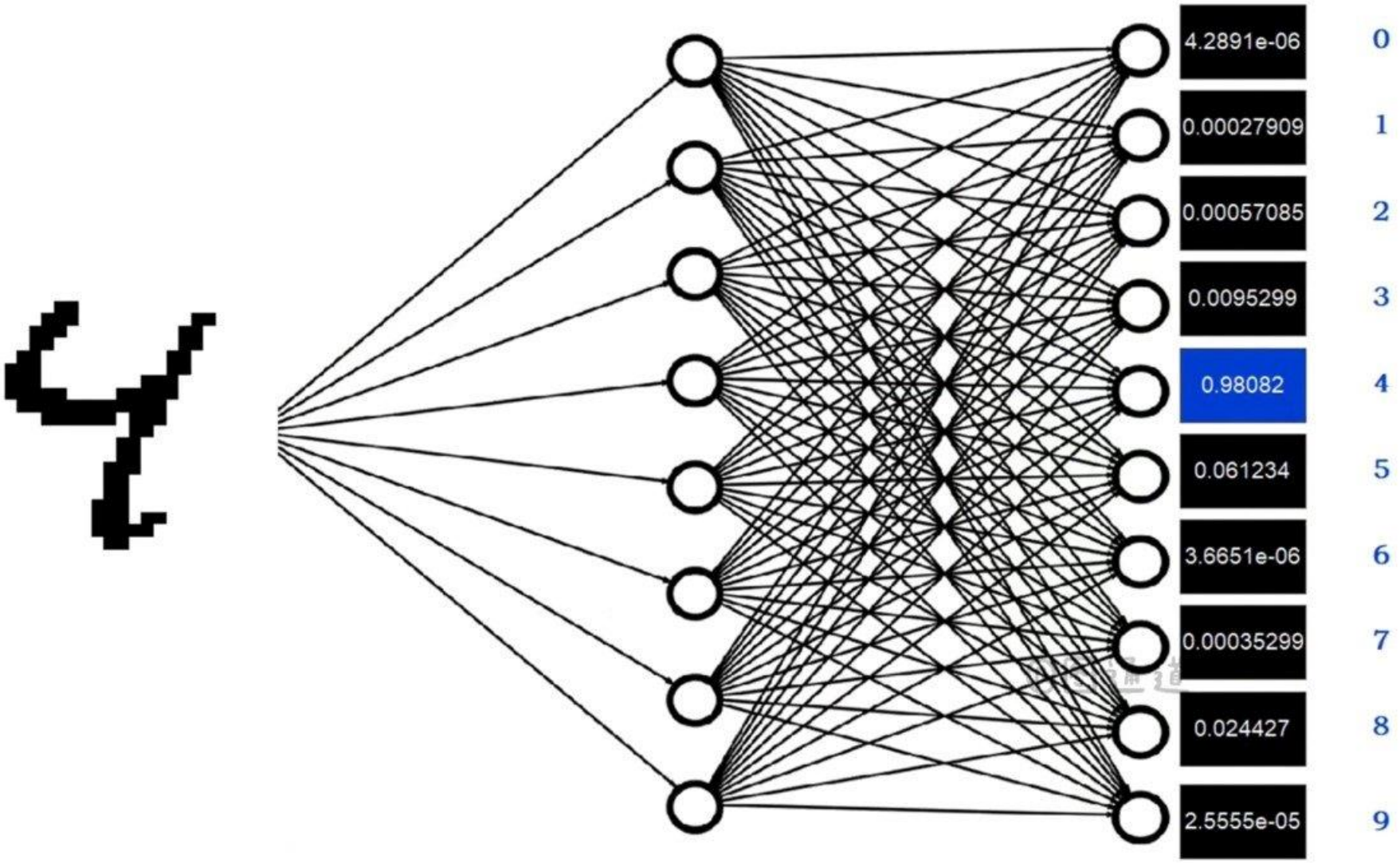
=1

$n^2 \times 1$



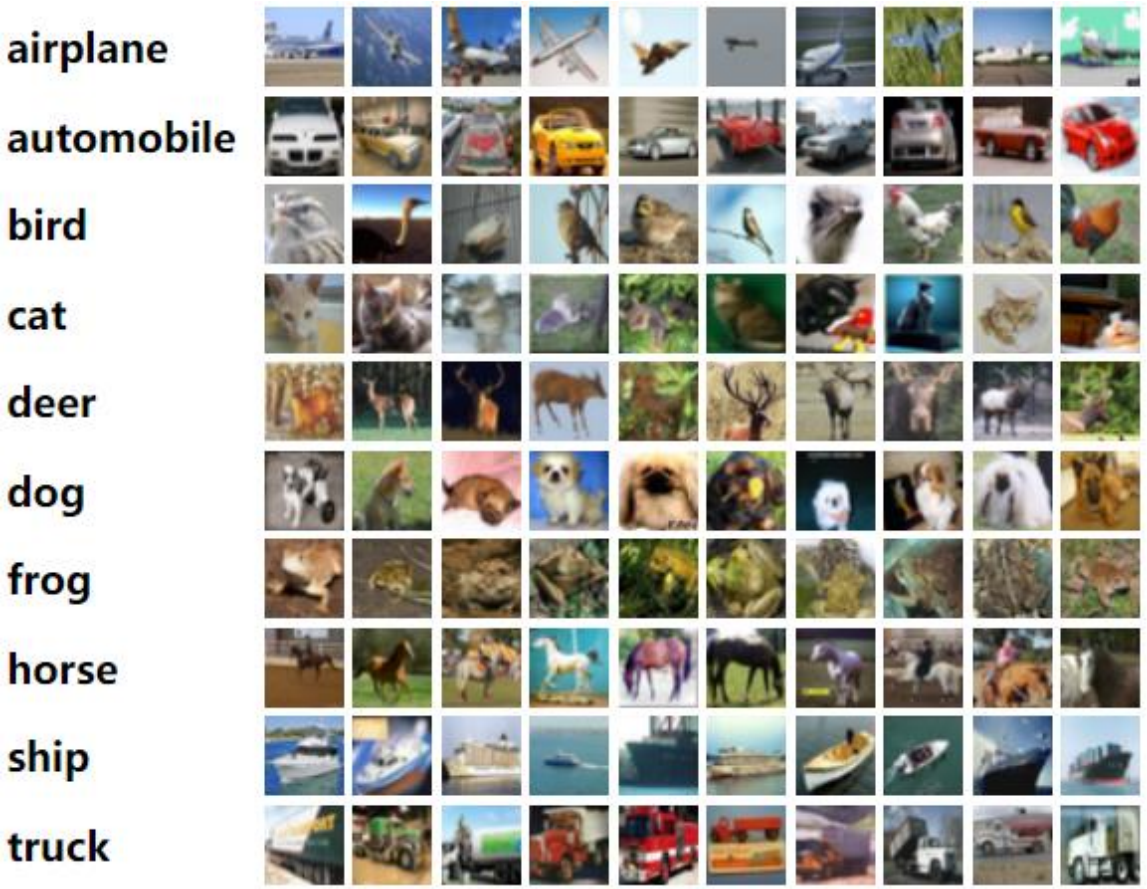
$n^2 - m - 10$

手写数字训练


4.2891e-06
0.00027909
0.00057085
0.0095299
0.98082
0.061234
3.6651e-06
0.00035299
0.024427
2.5555e-05
0
1
2
3
4
5
6
7
8
9

CIFAR10数据库训练

CIFAR10数据集训练



- CIFAR - 10 数据集包含 10 个不同类别的 60000 张彩色图像，训练集 50000 张图像用于训练，测试集10000 张。
- 图像的尺寸均为 32×32 像素，具有三个颜色通道（红、绿、蓝）。
- 涵盖飞机、汽车、鸟类、猫、鹿、狗、青蛙、马、船和卡车。